

Proiect TSS 2026 - Analiza și Testarea Aplicației Tax Calculator

Echipa de Proiect

5 mai 2026

Cuprins

1	Introducere	2
2	Configurația Sistemului	2
2.1	Hardware	2
2.2	Software	2
2.3	Versiuni Tool-uri	2
3	Documentația Strategiilor de Testare	2
3.1	Testarea Funcțională	2
3.2	Acoperirea Structurală (Structural Coverage)	2
3.2.1	Rezultate Acoperire	3
3.3	Testarea prin Mutație (Mutation Testing)	3
4	Analiza Complexității	3
5	Utilizarea Tool-urilor AI (GitHub Copilot)	3
6	Demo și Rularea Testelor (Video)	3
7	Concluzii	3
8	Referințe Bibliografice	4

1 Introducere

Acest proiect vizează implementarea și testarea riguroasă a unei aplicații de calcul fiscal, `Tax-Calculator`. Obiectivul principal este aplicarea metodelor de testare învățate în cadrul cursului de Testarea Sistemelor Software (TSS), incluzând testarea funcțională, analiza acoperirii structurale și testarea prin mutație.

2 Configurația Sistemului

2.1 Hardware

- Procesor: Intel Core i7 / AMD Ryzen 5 (sau echivalent)
- Memorie RAM: 16 GB
- Stocare: SSD NVMe

2.2 Software

- Sistem de Operare: Linux (Ubuntu 22.04 LTS / Debian)
- Limbaj de Programare: Python 3.13.2
- Virtualizare: Nu s-a utilizat mașină virtuală (rulare nativă în `venv`)

2.3 Versiuni Tool-uri

Tool	Versiune
Python	3.13.2
Pytest	8.1.1
Coverage.py	7.13.5
Radon (Complexity)	6.0.1
Mutmut (Mutation)	3.2.1 (estimat)

3 Documentația Strategiilor de Testare

3.1 Testarea Funcțională

Testarea funcțională a fost realizată utilizând tehnici de analiză a valorilor de frontieră și partiționare pe clase de echivalență. Detalii complete se regăsesc în `docs/functional_testing.md`.

3.2 Acoperirea Structurală (Structural Coverage)

Am urmărit obținerea unei acoperiri de 100% pentru instrucțiuni (Statement Coverage) și ramuri (Branch Coverage).

3.2.1 Rezultate Acoperire

- **Statement Coverage:** 100%
- **Branch Coverage:** 100%

3.3 Testarea prin Mutație (Mutation Testing)

Am evaluat calitatea testelor prin generarea de mutanți. Scorul obținut a fost de aproximativ 79% folosind `mutmut`.

4 Analiza Complexității

Funcția `calculate_annual_tax` prezintă o complexitate ciclomatică ridicată ($CC = 41$), ceea ce a impus o testare extrem de detaliată a tuturor ramificațiilor.

5 Utilizarea Tool-urilor AI (GitHub Copilot)

Am utilizat GitHub Copilot pentru accelerarea scrierii testelor. O analiză comparativă între testele manuale și cele generate de AI este prezentată în Tabelul 2.

Tabela 2: Comparatie Teste Manuale vs. AI

Aspect	Teste Manuale	GitHub Copilot
Eficiență	Scăzută	Foarte Ridicăta
Acoperire	Completă (bazată pe CFG)	Parțială (repetitivă)
Fiabilitate	Maximă	Necesită verificare manuală

6 Demo și Rularea Testelor (Video)

Demo-ul aplicației și procesul de rulare a testelor au fost înregistrate și sunt disponibile la următoarele link-uri:

- **Demo Aplicație:** https://youtube.com/demo_placeholder
- **Rulare Teste și Rapoarte:** https://youtube.com/tests_placeholder

7 Concluzii

Proiectul a demonstrat importanța utilizării unor instrumente automate de analiză a codului și a testelor. Deși complexitatea codului este mare, suita de teste dezvoltată asigură o robustețe ridicată a aplicației.

8 Referințe Bibliografice

1. Myers, G. J., "The Art of Software Testing", Wiley, 2011.
2. Documentation for Coverage.py: <https://coverage.readthedocs.io/>
3. GitHub Copilot: <https://github.com/features/copilot>